

3. Git Configuration Commands

Tumhare notes me `git config`, username/email aur config list/unset ke commands hain.

 pdf_08292026_152610

Command	Kya kaam karegi?	Deep Explanation	
<code>git config --list</code>	Saari Git configuration dikhata hai	Git ke currently available configuration values dekhta hai	
<code>git config --global --list</code>	Global configuration dikhata hai	User-level settings show karta hai	
<code>git config --global user.name "Tarun"</code>	Git username set karta hai	Future commits me author name ke liye use hota hai	
<code>git config --global user.email "email@example.com"</code>	Git email set karta hai	Commit ke author ke saath email associate hota hai	
<code>git config --local user.name "Tarun"</code>	Current repo ke liye username	Sirf current repository par apply hota hai	



Git Hub

<pre>git config --local user.email "email@example.com"</pre>	Current repo ka email	Sirf current repository ke liye
<pre>git config --global --unset user.name</pre>	Global username remove	Global username configuration delete karta hai
<pre>git config --global --unset user.email</pre>	Global email remove	Global email configuration delete karta hai
<pre>git config --global --show- origin --list</pre>	Setting kis file se aa rahi hai	Configuration ka source/location bhi dikhata hai

Global vs Local

</> **Bash**

```
git config --global user.name "Tarun"
```



</> **Bash**

```
git config --global user.name "Tarun"
```

Matlab:

Mere computer par Git ki default identity = Tarun

Lekin:

</> **Bash**

```
git config --local user.name "Company Name"
```

Matlab:

Sirf current project ke commits me ye identity use karo

4. Repository Initialize Commands

Notes me `git init` important command ke roop me hai.

 pdf_08292026_152610

Command	Kya kaam karegi?	Deep Explanation
<code>git init</code>	Current folder ko Git repository banata hai	<code>.git</code> hidden directory create hoti hai
<code>git init project-name</code>	New Git repository folder banata hai	Folder create karke usme Git initialize karta hai
<code>git status</code>	Repository ki current state	Git ko pata chalta hai kaunsi files changed/untracked hain

6. Git Add Commands

Notes me `git add` aur staging area ka concept repeatedly hai.

 pdf_08292026_152610

Command	Kya kaam karegi?	Deep Explanation	
<code>git add file.js</code>	Specific file stage	Sirf <code>file.js</code> next commit ke liye ready	
<code>git add .</code>	Current directory ki changes stage	Multiple changed/untracked files staging area me	
<code>git add -A</code>	All changes stage	Add, modify aur delete changes ko stage karta hai	
<code>git add -u</code>	Tracked files ki changes stage	Untracked files ko add nahi karta	

7. File ko Staging se Remove Karna

Agar galti se:

```
</> Bash
```

```
git add .
```



kar diya aur ek file stage nahi karni:

```
</> Bash
```

```
git restore --staged file.js
```



Command	Kya karegi?
<code>git restore --staged file.js</code>	File ko staging area se hataegi
<code>git restore --staged .</code>	Sabhi staged files ko unstage karegi

Important: Isse generally file ke actual modifications delete nahi hote; staging se selection remove hota hai.



8. Commit Commands

Notes me `git commit -m` repeatedly use hua hai.

 pdf_08292026_152610

Command	Kya kaam karegi?	Deep Explanation
<code>git commit -m "message"</code>	Staged changes ka snapshot save	Current staged state ko local repository me commit karta hai
<code>git commit -am "message"</code>	Tracked modified files ko stage + commit	New/untracked files automatically add nahi hoti
<code>git commit --amend</code>	Last commit modify	Last commit me changes/message modify kar sakte ho
<code>git commit --amend --no-edit</code>	Last commit me changes add	Commit message same rakhta hai

9. Git Log / History Commands

Tumhare notes me commit history aur commit hash se reset ka concept bhi hai.

 pdf_08292026_152610

Command	Kya kaam karegi?
<code>git log</code>	Complete commit history
<code>git log --oneline</code>	Short commit history
<code>git log --oneline --all</code>	Sab branches ki compact history
<code>git log --graph</code>	Branch/merge history graph
<code>git log --oneline --graph --all</code>	Graph + all branches
<code>git show <commit-hash></code>	Particular commit ki details
<code>git diff</code>	Unstaged changes
<code>git diff --staged</code>	 staged changes

10. Git Clone — GitHub se Project Lana

Notes me `git clone URL` diya gaya hai.  pdf_08292026_152610

Command	Kya kaam karegi?
<code>git clone <URL></code>	GitHub repository ki complete copy local machine par
<code>git clone <URL> folder-name</code>	Repository ko specified folder name me clone
<code>git clone --branch dev <URL></code>	Specific branch clone

Example:

 **Bash**

```
git clone https://github.com/user/project.git
```



11. Remote / GitHub Commands

Tumhare notes ke pages 4 aur 10 me remote commands clearly covered hain.

 pdf_08292026_152610 +1

`origin` kya hai?

`origin` GitHub repository ka **default remote name** hota hai.

Example:

 **Bash**

```
git remote add origin https://github.com/tarun/project.git
```

Yahan:

Command	Kya kaam karegi?	Deep Explanation	
<code>git remote</code>	Remotes ke names	Connected remote names	
<code>git remote -v</code>	Remote URLs	Fetch/push URLs show	
<code>git remote add origin <URL></code>	GitHub remote add	Local repository ko GitHub repository se connect	
<code>git remote remove origin</code>	Remote remove	<code>origin</code> connection delete	
<code>git remote set-url origin <URL></code>	Remote URL change	Existing origin ko new GitHub URL deta hai	
<code>git remote rename origin upstream</code>	Remote name change	Remote ka naam change	

12. Push Commands

Tumhare notes me `git push -u origin main`, `git push origin main` etc. hain.

 pdf_08292026_152610

Command	Kya kaam karegi?
<code>git push</code>	Local commits remote par bhejta hai
<code>git push origin main</code>	Local <code>main</code> branch ko GitHub ke <code>main</code> par push
<code>git push -u origin main</code>	Push + upstream tracking set
<code>git push origin dev</code>	<code>dev</code> branch GitHub par push
<code>git push --all origin</code>	All local branches push
<code>git push --tags</code>	Tags remote par push

`-u` ka important meaning

First time:

`</> Bash`

```
git push -u origin main
```

`-u` upstream tracking establish karta hai.

Uske baad normally:

`</> Bash`

```
git push
```

kaafi hota hai.

13. Pull Commands

Notes me `git pull origin main` bhi hai.  pdf_08292026_152610

Command	Kya kaam karegi?
<code>git pull</code>	Remote changes lekar current branch me integrate
<code>git pull origin main</code>	GitHub ke main se changes pull
<code>git pull --rebase</code>	Changes ko rebase approach se integrate

Command	Kya hota hai?
<code>git fetch</code>	Remote se latest commits ki information lata hai
<code>git fetch origin</code>	Origin se updates lata hai
<code>git fetch --all</code>	Sab remotes se updates
<code>git pull</code>	Fetch + integrate
<code>git pull --rebase</code>	Fetch + rebase

Difference

```
git fetch
```

sirf information update karta hai.

```
git pull
```



information lane ke baad current branch ko update/integrate bhi karta hai.

15. Branch Commands

Tumhare notes me branching ka bada section hai.  pdf_08292026

Branch ka matlab:

Project ke main code se independent development line.

Example:

```
main
|
├─ login-feature
|
├─ payment-feature
|
└─ admin-feature
```

<code>git branch</code>	Local branches show
<code>git branch -a</code>	Local + remote branches
<code>git branch -r</code>	Remote branches
<code>git branch dev</code>	New <code>dev</code> branch create
<code>git branch -d dev</code>	Merged branch delete
<code>git branch -D dev</code>	Force branch delete
<code>git branch -M main</code>	Current branch ka naam <code>main</code>
<code>git checkout dev</code>	<code>dev</code> branch par switch
<code>git checkout -b dev</code>	Branch create + switch
<code>git switch dev</code>	Branch switch
<code>git switch -c dev</code>	Create + switch

16. `checkout -b` vs `switch -c`

Old/common Git style:

`</>` **Bash**

```
git checkout -b feature
```

Modern clearer style:

`</>` **Bash**

```
git switch -c feature
```

Dono ka basic purpose:

```
new branch create  
+  
us branch par switch
```

```
main
 \
  dev
  \
    merged into main
```

Commands

Command	Kya kaam karegi?
<code>git merge dev</code>	<code>dev</code> ko current branch me merge
<code>git merge main</code>	<code>main</code> ko current branch me merge
<code>git merge --abort</code>	Conflict wali merge cancel
<code>git merge --no-ff dev</code>	Explicit merge commit create karne ki koshish

Golden rule

Agar `dev` ko `main` me merge karna hai:

</> Bash

```
git switch main
git pull origin main
git merge dev
git push origin main
```

18. Merge Conflict

Suppose:

```
main → index.html me heading = Hello
```

```
dev → index.html me heading = Welcome
```

Dono branches ne **same line** change kar di.

Merge:

```
</> Bash
```

```
git merge dev
```

Git conflict de sakta hai.

File me:

```
<<<<<< HEAD
Hello
=====
Welcome
>>>>>> dev
```

Tum manually decide karoge:

```
Welcome
```

Phir:

```
</> Bash

git add .
git commit -m "resolve merge conflict"
```

Agar merge cancel karna ho:

```
git merge --abort
```

Agar merge cancel karna ho:

```
</> Bash

git merge --abort
```



Notes ke later pages me merge/conflict handling aur uske baad `add`, `commit`, `push` ka flow diya gaya hai.  pdf_08292026_152610

Ye commands carefully samjho.

Command	Kya karegi?	Risk
<code>git reset HEAD~1</code>	Last commit undo; changes working directory me	Medium
<code>git reset --soft HEAD~1</code>	Commit undo, changes staged rahengi	Low
<code>git reset --mixed HEAD~1</code>	Commit undo, changes unstaged	Medium
<code>git reset --hard HEAD~1</code>	Commit + working changes remove	High
<code>git reset <hash></code>	HEAD ko particular commit par move	Depends
<code>git reset --hard <hash></code>	Particular commit par code bhi reset	High

20. HEAD kya hai?

Git me:

HEAD

normally current checked-out commit/branch ko point karta hai.

Example:

```
A → B → C
      ↑
    HEAD
```

</> **Bash**

```
git reset HEAD~1
```

ke baad:

```
A → B
      ↑
```



ke baad:

```
A → B
  ↑
HEAD
```

HEAD~1 = current HEAD se ek commit peeche.

```
HEAD~1
HEAD~2
HEAD~3
```

means:

```
1 commit back
2 commits back
3 commits back
```

21. Revert vs Reset

Ye dono confuse mat karna.

Command	Meaning
<code>git reset</code>	Current branch ka pointer move kar sakta hai
<code>git revert</code>	Existing commit ko undo karne ke liye new commit banata hai

Example:

 **Bash**

```
git revert abc123
```



Suppose:

A → B → C



C ko revert karne ke baad:



A → B → C → C-revert



History safe rehti hai.

Isliye already shared/pushed commits ko undo karne ke liye `revert` generally safer hota hai.

22. Reflog — Galti ke baad Recovery

Related important command:

`</> Bash`

`git reflog`



Ye Git ke HEAD/reference movements ka record dikhata hai.

Example:

```
abc123 HEAD@{0}: reset  
def456 HEAD@{1}: commit
```



```
def456 HEAD@{1}: commit
```

Agar galti se reset kar diya:

```
</> Bash
```

```
git reset --hard HEAD~2
```

to `reflog` se purane commit/reference ko locate karke recovery possible ho sakti hai.

Example:

```
</> Bash
```

```
git reflog
```

Then:

```
</> Bash
```

```
git reset --hard <old-hash>
```



Reflog ko Git ka recovery/history safety net samajh sakte ho.

23. Stash Commands

Ye tumhare notes me clearly prominent nahi hai, lekin Git ke related **must-know** commands hain.

Suppose tum `dev` branch par kaam kar rahe ho:

```
10 files modified
```

Lekin suddenly `main` par jaana hai.

Commit nahi karna chahte.

```
</> Bash  
  
git stash
```

Changes temporarily store ho jayengi.

Changes temporarily store ho jayengi.

Command	Kya kaam karegi?
<code>git stash</code>	Current uncommitted changes temporarily save
<code>git stash list</code>	Stashes show
<code>git stash pop</code>	Latest stash restore + stash remove
<code>git stash apply</code>	Latest stash restore but stash list me rakhta hai
<code>git stash drop</code>	Stash delete
<code>git stash clear</code>	All stashes remove

Flow:

</> Bash

```
git stash  
git switch main
```

Baad me:

</> Bash

```
git switch dev  
git stash pop
```

24. File Management Commands

Command	Kya kaam karegi?
<code>git rm file.js</code>	File delete + Git deletion stage
<code>git rm --cached file.js</code>	Git tracking se remove, local file retain
<code>git mv old.js new.js</code>	File rename/move with Git tracking
<code>git clean -n</code>	Untracked files kya delete honge preview
<code>git clean -f</code>	Untracked files delete
<code>git clean -f</code> bhi carefully use karna.	

25. .gitignore

Project me kuch files GitHub par nahi bhejni hoti:

```
node_modules/  
.env  
*.log  
dist/
```

.gitignore me:

```
</> gitignore  
  
node_modules/  
.env  
dist/  
*.log
```

Then:

26. Tag Commands

Release/version mark karne ke liye:

Command	Kya kaam karegi?
<code>git tag</code>	Existing tags show
<code>git tag v1.0.0</code>	Lightweight tag
<code>git tag -a v1.0.0 -m "Release 1.0"</code>	Annotated tag
<code>git show v1.0.0</code>	Tag details
<code>git push origin v1.0.0</code>	Specific tag push
<code>git push origin --tags</code>	All tags push
<code>git tag -d v1.0.0</code>	Local tag delete

27. GitHub Credential Commands — Tumhare Page 18

Tumhare last page me **Windows Credential Manager** aur Git credential configuration ke commands hain.  pdf_08292026_152610

Notes me:

 **Bash**



```
git config --global --unset credential.helper
```

aur:

 **Bash**



```
cmdkey /list
```

aur GitHub credential delete karne ke liye `cmdkey /delete ...` type commands diye gaye hain.  pdf_08292026_152610

Important table

Command	Kya kaam karegi?
<code>git config --global --unset credential.helper</code>	Global credential-helper setting remove
<code>git config --global credential.helper</code>	Current credential helper setting check
<code>cmdkey /list</code>	Windows Credential Manager me stored credentials list
<code>cmdkey /delete:<target></code>	Particular Windows credential remove
<code>git credential-manager</code>	Git Credential Manager ke operations ke liye
<code>git credential-manager erase</code>	Stored Git credential remove karne ke workflow me use

Important: `cmdkey` Git command nahi hai; ye Windows ka credential-management command hai.



30. GitHub se Latest Code Lane ka Flow

Team project me:

</> Bash

```
git switch main  
git pull origin main
```

Feature branch:

</> Bash

```
git switch -c login-feature
```

Code karo:

</> Bash

```
git status  
git add .  
git commit -m "added login feature"
```



GitHub:

</> Bash

```
git push -u origin login-feature
```



Phir GitHub par Pull Request create karke `login-feature → main` merge kar sakte ho.

31. Branch + Merge Complete Flow

Ye interview/project dono ke liye yaad rakho:

</> Bash

```
git switch main
git pull origin main

git switch -c feature

# code changes
```



```
# code changes
```

```
git add .
```

```
git commit -m "feature added"
```

```
git push -u origin feature
```

Phir merge:

```
</> Bash
```

```
git switch main
```

```
git pull origin main
```

```
git merge feature
```

```
git push origin main
```

32. Most Important Commands — Priority Wise

Level 1 — Must Know

Command	Purpose
<code>git --version</code>	Git version
<code>git config</code>	Git configuration
<code>git init</code>	Repository create
<code>git status</code>	Current state
<code>git add .</code>	Stage changes
<code>git commit -m</code>	Commit
<code>git log</code>	History
<code>git clone</code>	Repository download
<code>git remote -v</code>	Remote check



Git Hub

<code>git clone</code>	Repository download
<code>git remote -v</code>	Remote check
<code>git remote add origin</code>	GitHub connect
<code>git push</code>	GitHub par upload
<code>git pull</code>	GitHub se latest changes
<code>git branch</code>	Branch management
<code>git switch</code>	Branch change
<code>git merge</code>	Branch merge

 Level 2 — Strong Git Knowledge

Command	Purpose
<code>git fetch</code>	Remote updates fetch
<code>git diff</code>	Changes compare
<code>git restore</code>	Changes restore
<code>git reset</code>	HEAD/history move
<code>git revert</code>	Commit safely undo
<code>git reflog</code>	Recovery/reference history
<code>git stash</code>	Temporary changes save
<code>git cherry-pick</code>	Particular commit apply
<code>git tag</code>	Version mark



<code>git tag</code>	Version mark
<code>git remote set-url</code>	Remote URL change
<code>git merge --abort</code>	Merge cancel

33. Ek line me poora Git yaad karo

```
git init
↓
git status
↓
git add .
↓
git commit -m "message"
↓
git remote add origin <github-url>
↓
git branch -M main
```



git restore

Command	Kya kaam karegi?	DeePLY	
<code>git restore file.js</code>	Working directory ki changes hataegi	<code>file.js</code> ko last committed/staged state ke according restore karti hai	
<code>git restore .</code>	Current folder ki unstaged changes hataegi	Multiple modified files ko previous state me la sakti hai	
<code>git restore --staged file.js</code>	File ko staging se hatayegi	File ki changes delete nahi hoti , sirf staging area se remove hoti hai	
<code>git restore --staged .</code>	Sabhi files unstage	Staging area ko previous HEAD state ke according restore karta hai	
<code>git restore --source=HEAD file.js</code>	HEAD version restore	Current commit (<code>HEAD</code>) wali file restore karta hai	
<code>git restore --source=<commit> file.js</code>	Specific commit se file restore	Purane commit ki file version current working directory me la sakta hai	